CIDR Math Reference

Built from the run-2 CIDR drill. A reader-friendly version of `mathCIDR.md`: the formulas, the step-by-step workflow, and the traps that caused real mistakes.

1. The core idea: locked bits and free bits

An IPv4 address is **32 bits**. The prefix number (`/16`, `/24`, ...) is how many of those bits are **locked** as the network part. Every bit that is not locked is **free**, and free bits are what give you addresses.

> **Free bits = 32 - prefix**
>
> **Addresses = $2^{\text{(free bits)}}$**

CIDR	Free bits (32 - prefix)	Addresses (2^{free})
/8	24	16,777,216
/16	16	65,536
/23	9	512
/24	8	256
/28	4	16
/32	0	1

Smaller prefix number = more free bits = a bigger block. `/32` is exactly one address.

2. Octet locking

An address has four octets (`A.B.C.D`), 8 bits each. When the prefix lands on a multiple of 8, whole octets are locked:

Prefix	Locked octets	Pattern for 10.0.x.x
/16	first two	`10.0.X.X`
/24	first three	`10.0.X.Y`

> **KEY POINT**
>
> Locked octets are numbers you **choose**, not a fixed default. Inside a `10.0.0.0/16` VPC, a subnet can use *any* third octet (`10.0.5.0/24`, `10.0.7.0/24`, `10.0.8.0/24`, ...) as long as it is (1) inside the VPC's own locked range and (2) not already claimed by another subnet in that VPC. Nothing about `0` is special; it is only "correct" when nothing else occupies it.

3. Finding the exact range for any prefix

This works for **any** prefix, not just clean `/8`, `/16`, `/24` boundaries. Do the steps in order. The right-hand column runs them on `172.16.50.128/25`.

#	Step	How	Example: 172.16.50.128/25
1	**Full octets locked**	prefix ÷ 8, whole-number part	25 ÷ 8 = 3 (whole part) -> 3 octets locked
2	**Bits locked in the next octet**	prefix mod 8 (the remainder)	25 mod 8 = 1 -> 1 bit locked in the 4th octet
3	**Free bits in that octet**	8 - remainder	8 - 1 = 7
4	**Step size**	$2^{(\text{free bits in that octet})}$. Valid starting values in the octet are multiples of this.	$2^7 = 128$
5	**Binary check**	Write the octet value in binary, split locked / free, and run the free bits from all 0 to all 1.	128 = `1 | 0000000`. Free bits `0000000` to `1111111` -> 128 to 255
6	**Write the range**	start through **start + step - 1** in that octet, then 0 to 255 in every fully-free octet after it.	128 + 128 - 1 = 255 -> **172.16.50.128 - 172.16.50.255**

Edge case: if the remainder in step 2 is 0 (for example `/16` or `/24`), there is no partially-locked octet. Every octet after the locked ones is simply free, `0` to `255`.

SANITY CHECK, EVERY TIME

Total free bits (**32 - prefix**) must equal **(free bits in the partial octet) + (8 × number of fully-free octets after it)**.

For /25: 32 - 25 = 7, and 7 + (8 × 0) = 7. It matches, so the answer is trustworthy. If it doesn't match, one earlier step is wrong. Go back and find it before trusting the answer.

4. The two traps that caused real mistakes

TRAP 1: THE CALCULATOR DECIMAL

Dividing gives a decimal, for example `20 ÷ 8 = 2.5`. The digits after the decimal point are **not** the remainder, so `2.5` does not mean "remainder 5."

Fix: multiply the decimal part by 8. `0.5 × 8 = 4`, so the remainder is 4. (Or just compute `20 mod 8 = 4` directly.)

TRAP 2: THE OFF-BY-ONE AT THE TOP OF THE RANGE

The top of the range is **start + step - 1**, **not** start + step. Counting *n* values from a starting point lands on *start + (n - 1)*.

Check: a /25 starting at 128 has step 128. Top = 128 + 128 - 1 = **255**, never 256 (an octet cannot exceed 255).

5. Worked examples

Example A: `172.16.50.128/25` (from the drill)

Walked through in the table above. Result: **172.16.50.128 - 172.16.50.255** (128 addresses).

Example B: `10.0.16.0/20` (same method, a non-/8 prefix in the third octet)

Step	Work	Result
1. Full octets	20 ÷ 8 = 2 (whole part)	2 octets locked: `10.0`
2. Remainder	20 mod 8 = 4	4 bits locked in the 3rd octet
3. Free bits	8 - 4	4 free bits in the 3rd octet
4. Step size	2^4	16 (valid third octets: 0, 16, 32, ...)
5. Binary	16 = `0001 | 0000`	free bits run `0000` to `1111` -> 16 to 31
6. Range	16 + 16 - 1 = 31, then fourth octet 0 to 255	**10.0.16.0 - 10.0.31.255**
Sanity	32 - 20 = 12	4 + (8 × 1) = 12. Matches.

6. Overlap: check the actual range, not the slash number

Two blocks overlap if their **address ranges intersect**. Never judge it by how similar or different the prefix numbers look. Get both ranges with the method above, then line them up:

```
/26:   .64 ---- .127
/25:               .128 ------------- .255     -> no overlap (adjacent, not shared)

/26:                        .192 ----- .255
/25:        .128 ---------------------- .255     -> overlap (fully contained)
```

Same prefix sizes and same base network (`10.20.30.x`). Only the starting octet moved, and that alone flipped it from separate to fully contained.

> **THE /23 TRAP**
>
> A `/23` spans **two consecutive /24 blocks** (an even/odd pair). `10.0.4.0/23` = `10.0.4.0/24` + `10.0.5.0/24`, i.e. `10.0.4.0` to `10.0.5.255`.
>
> So `10.0.5.0/24` is not next to it. It is **entirely inside** it. Same pattern as a `/8` fully containing every `/16` that starts with the same first octet.

7. Two different problems, two different octets

Problem	What to change	Example
Fit a subnet inside an existing VPC, distinct from sibling subnets	Keep the VPC's locked octets identical. Choose a free *next* octet.	`10.0.0.0/16` VPC: vary the **third** octet -> `10.0.5.0/24`, `10.0.7.0/24`
Make two separate VPCs non-overlapping (e.g. before peering)	Change one of the VPC's *own* locked octets. Changing the prefix length alone does **not** fix it.	`10.0.0.0/16` -> `10.1.0.0/16`: change the **second** octet

Why prefix length alone fails: `10.0.0.0/16` -> `10.0.0.0/8` makes the overlap *worse*. It becomes a superset, not a disjoint block.

One-page cheat sheet

Free bits = 32 - prefix

Addresses = $2^{\text{(free bits)}}$

Step	Do this
1	Full octets locked = prefix ÷ 8 (whole part)
2	Remainder = prefix mod 8 (bits locked in the next octet)
3	Free bits in that octet = 8 - remainder
4	Step size = $2^{\text{(free bits in that octet)}}$
5	Binary check: free bits go all 0 to all 1
6	Range = start to **start + step - 1**, then .0 to .255 on later octets
Check	32 - prefix = (free bits in partial octet) + 8 × (fully-free octets after it)

REMEMBER

- **Calculator trap:** `20 ÷ 8 = 2.5` does not mean remainder 5. Use mod, or 0.5 × 8 = 4.
- **Off-by-one:** top of range is start + step **- 1**.
- **Overlap:** compare the actual ranges, never the slash numbers.
- **/23 trap:** a /23 is two /24s, so a /24 inside it is contained, not adjacent.
- **VPC vs. VPC overlap:** change a locked octet. A different prefix length alone won't separate them.